IL2234

Digital Systems Design and Verification using Hardware Description Languages

Project Milestone 2

Register File and Memory

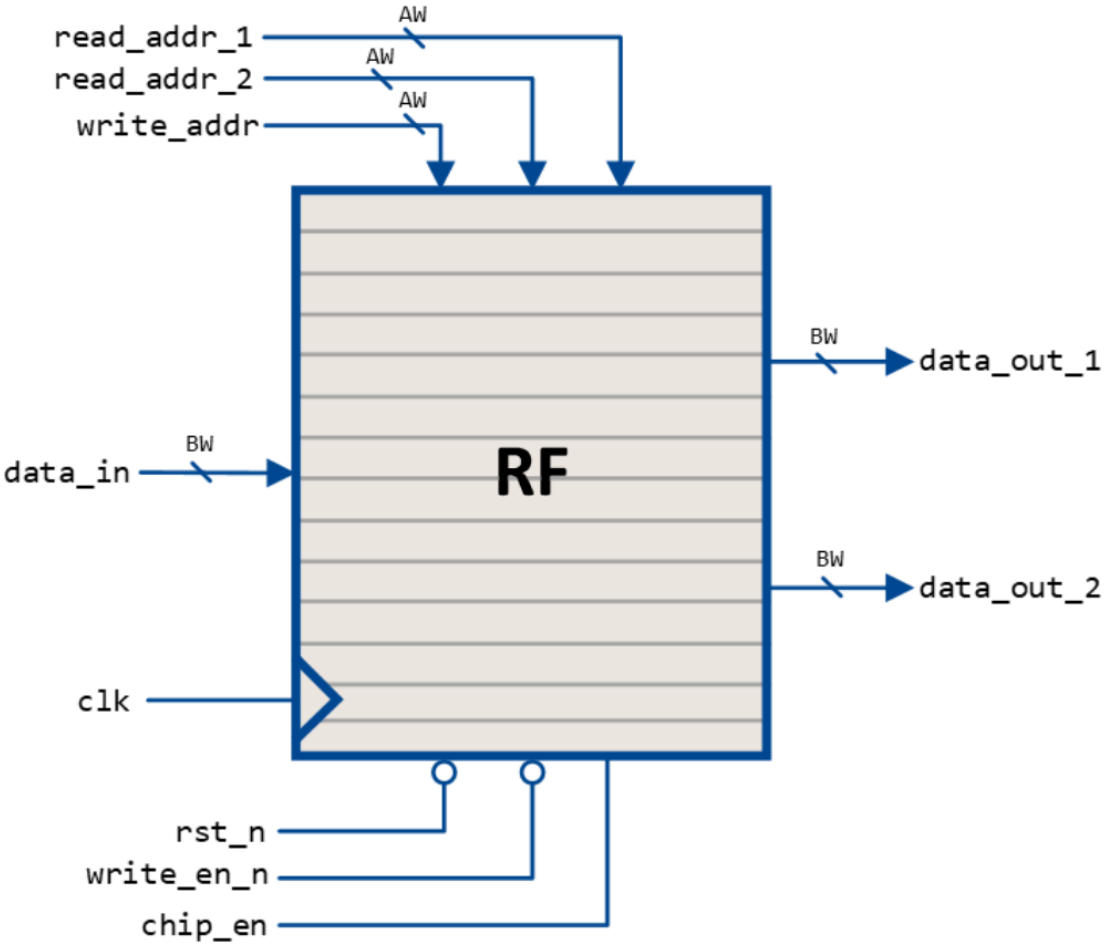
Part 1: Register File (RF)

Introduction

In this milestone, we will build the second component of our microprocessor: the Register File (RF). We will design and model the RF circuit based on the SystemVerilog modelling concepts learned in the class. Additionally, we will develop a simple testbench to test the basic functionality of the RF.

Overview

Below is a simple diagram of the RF



The inputs and outputs are described in the table below:

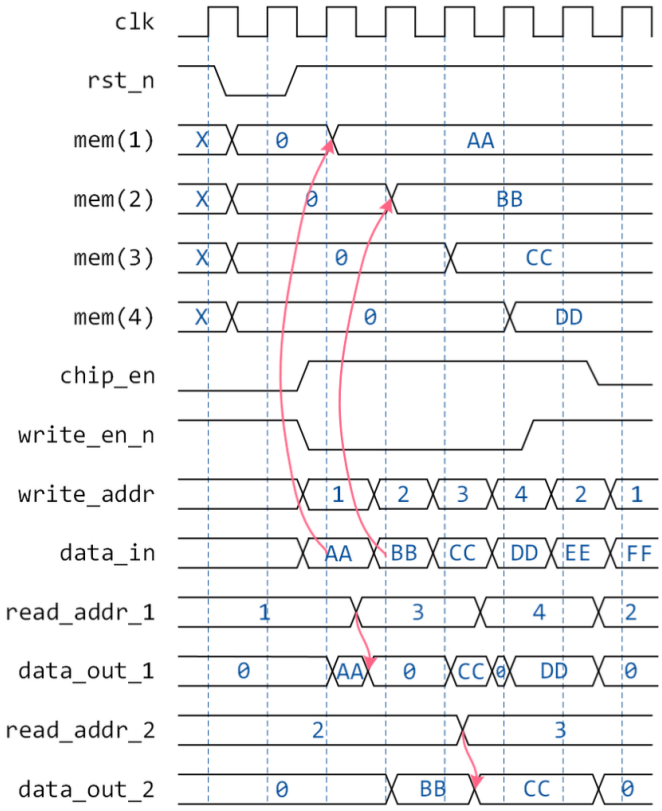
Name	Direction	Type	Bitwidth	Description
clk	Input	logic	1	Clock input
rst_n	Input	logic	1	Asynchronous active low reset signal
data_in	Input	logic	BW [1]	Data input port
data_out_1	Output	logic	BW [1]	Data output port 1
data_out_2	Output	logic	BW [1]	Data output port 2
read_addr_1	Input	logic	$clog2(DEPTH) [2]	Address for output port 1
read_addr_2	Input	logic	$clog2(DEPTH) [2]	Address for output port 2
write_addr	Input	logic	$clog2(DEPTH) [2]	Address for input port
write_en_n	Input	logic	1	Active low write enable
chip_en	Input	logic	1	Active high chip enable

The RF behaviour is defined according to the following table:

Function	clk	rst_n	chip_en	write_en_n	Memory	data_out_1	data_out_2
Reset	X	0	X	X	mem(all) = 0	0	0
Standby	X	1	0	X	No change	0	0
Read	X	1	1	1	No change	mem(read_addr_1)	mem(read_addr_2)
Write		1	1	0	mem(write_addr)= data_in	mem(read_addr_1)	mem(read_addr_2)

Address 0 is hardwired to zero: writes to it are ignored and reads always return 0.

A timing diagram showing a typical operation of the register file is shown below:



Tasks

1. Design the RF according to the specifications presented earlier. Write your code in System Verilog using **only synthesizable** constructs. Follow the same naming for the signals indicated in the diagrams and tables.

2. Write a testbench that generates read and writes on the RF. Use randomised data for the data and addresses. Use the printing functions to display the contents of the RF in the terminal and check against the expected results.

Deliverables

1. Submit the RTL design and testbench(es) on your KTH Github repository in *riscv/rf*

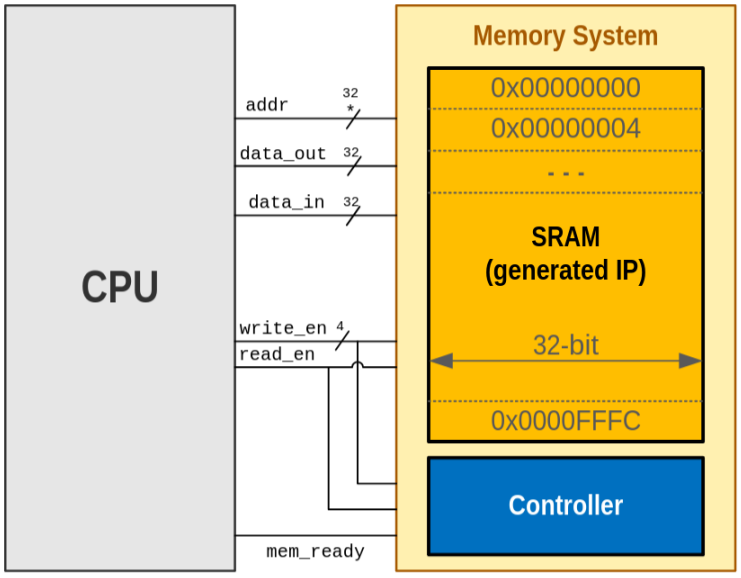
Submission date: *before the examination*

Individual submission: *same content is allowed within a group*

Part 2: Memory

Introduction

In this milestone, we will generate the SRAM that will be used as the main memory for data and instructions of our CPU design. We will use the Xilinx IP generator to generate a memory. Additionally, we will design a simple control logic to generate the flags which indicate to the CPU's controller that the memory operation is finished.

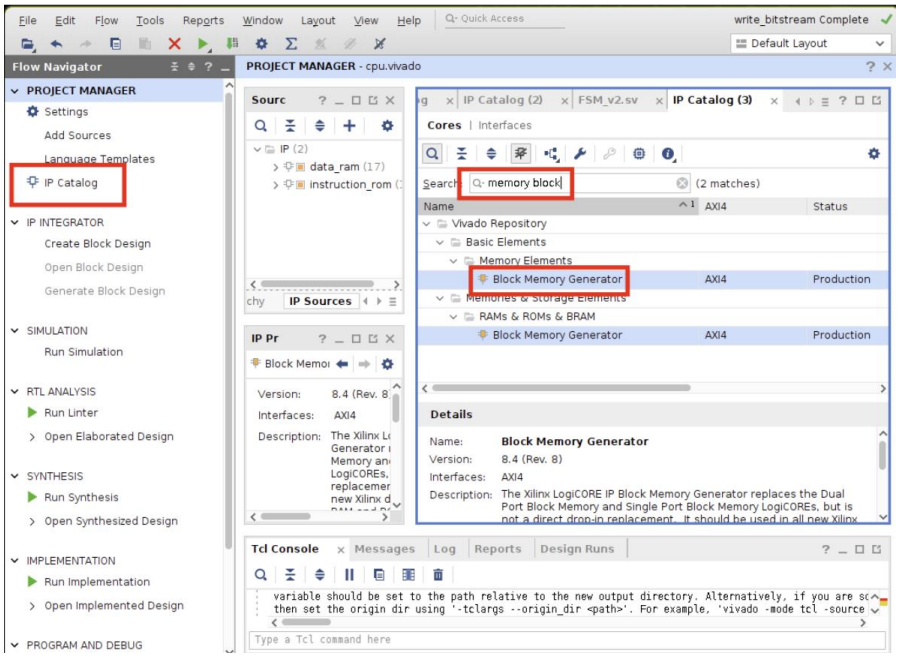


* The CPU address bus is 32 bits, but since we are using a 32-bit byte-addressable memory, the lower two bits of the address are used to generate the `write_en` signal (byte select), resulting in the "effective" address width being 30 bits.

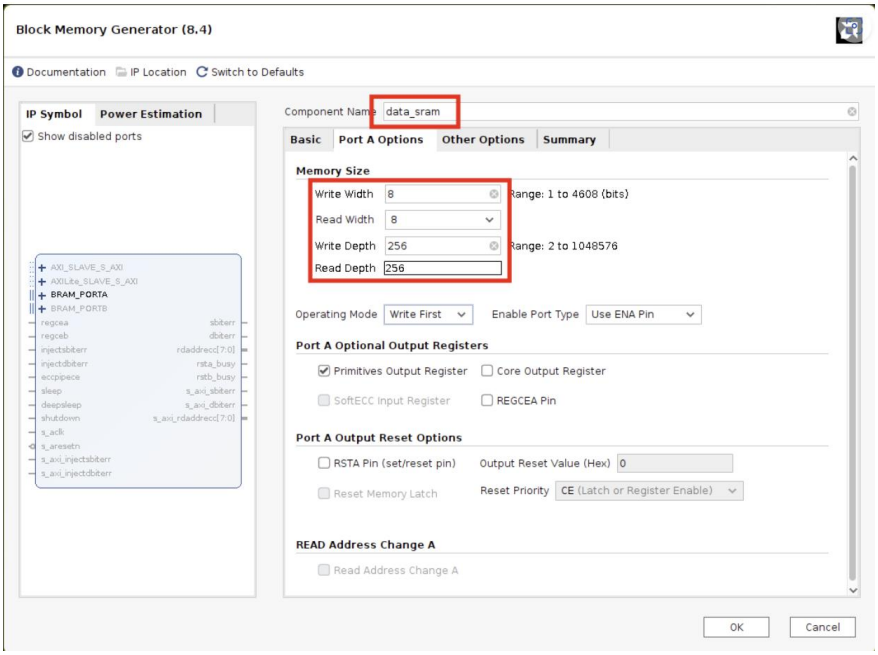
IP generation

Open the *IP catalogue* and search for Block Memory

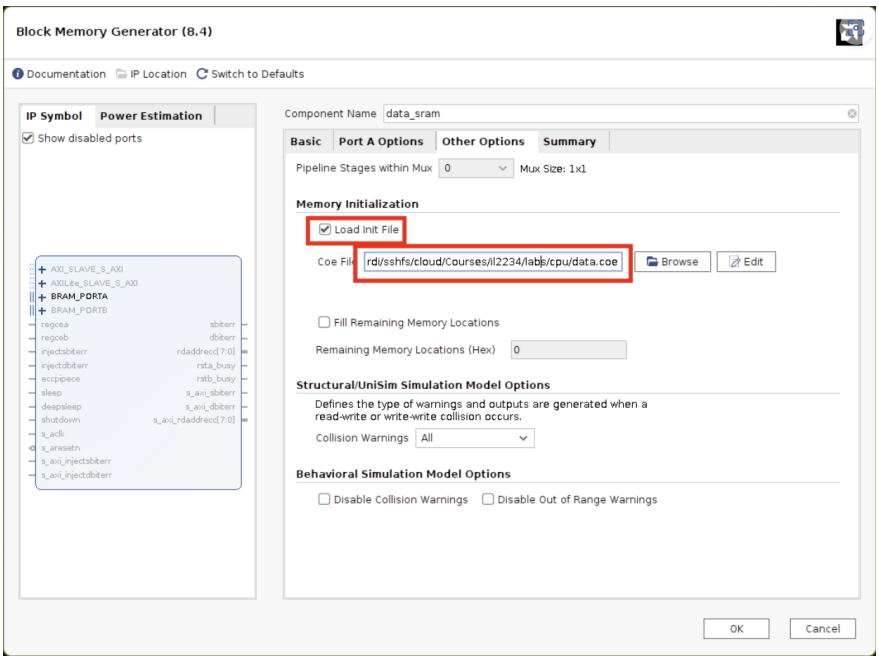
Choose the type of memory that you want to create (SRAM, ROM, etc.)



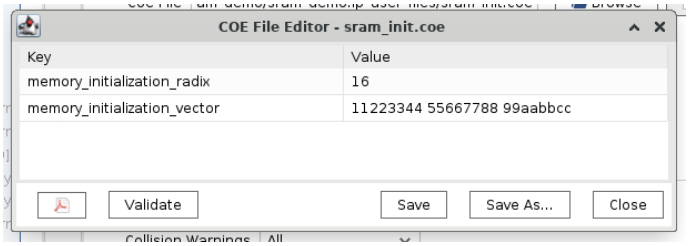
In *Port A Options*, select the size according to your design. Note that the depth refers to the number of memory rows; Vivado automatically calculates the address width as `clog2(DEPTH)`



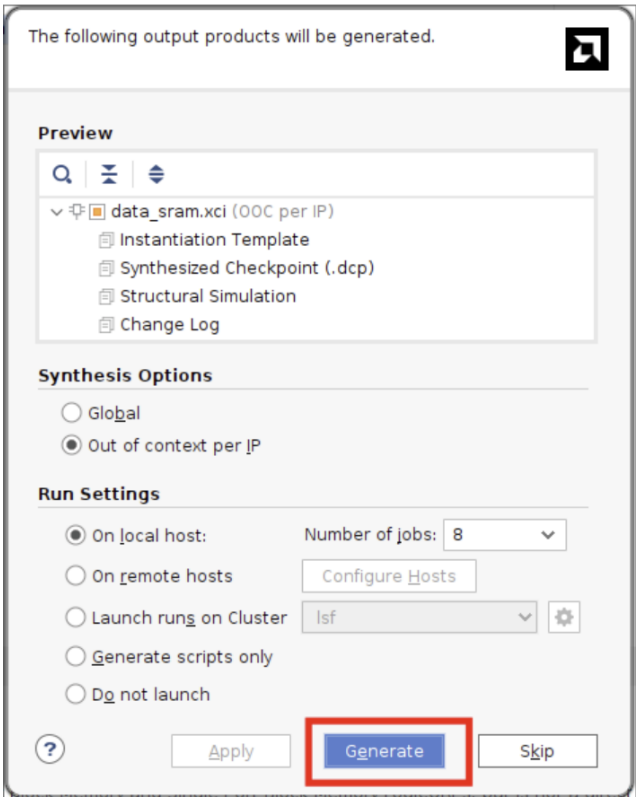
In *Other Options*, select the COE (Coefficient) and generate an initialization file with dummy data, which you will use to test the SRAM behaviour.



This is an example of dummy data in the first 3 addresses.



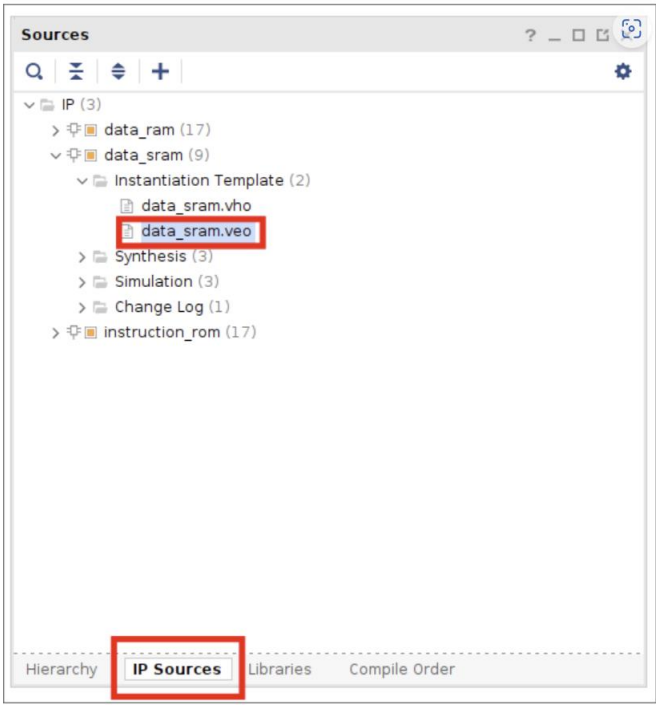
Click *OK*, and in the next window, click *Generate*.



You should now see your IP under *Sources->IP Sources*.

If you expand the generated block, you should see several files, among them instantiation templates.

Open the `.veo` and you will see an instantiation template for Verilog/SystemVerilog.



Tasks

1. Generate a SRAM block with the following parameters:
   - Data width: 32-bit
   - Depth: 16384 (corresponding to 64 KiB)
   - Output register
   - Byte-wise write enabled (byte size = 8 bit)
2. Create an RTL module that includes the generated SRAM macro and a simple state machine that generates the mem_ready flag. Note that this flag should be 1 when the output data is valid (for loads) and when the memory has been successfully written (for stores).
3. Create a testbench to test the different functionalities of the memory (read, write, byte-wise writes). Use this testbench to verify the timing of the SRAM regarding the enable and mem_ready signals. *You may want to use Vivado Simulator for this task.*

Deliverables

1. Submit the RTL design (not the generated SRAM) and testbench(es) on your KTH Github repository in *riscv/memsys*

Submission date: *before the examination*

Individual submission: *same content is allowed within a group*